Advanced Programming

Welcome to the course in Advanced Programming!

This course focuses on practical and industry-adapted functional programming. Languages such as C#, Java, JavaScript, or Python are not strictly functional languages, but all support functional principles to varying degrees and combine them with object orientation.

In this course, C# is used, which has strong support for functional programming. Advanced C# programming combines functional and object-oriented programming in an elegant way, and gives a deeper understanding to moderns design patterns.

Starting off the course, in this document I want to give you an overall comparison to the Functional programming paradigm and the Object Oriented programming paradigm and why the Hybrid Approach has become so popular

Code repository
https://github.com/orgs/SEIDOAB-ADFP/repositories

Table of content

Table of content 2
The Functional Programming Paradigm 3
Origins of Functional Programming 3
Advantages of Functional Programming 3
C# and Functional Programming Concepts 4
Some C# code examples 4
The Object-Oriented Programming Paradigm 7
Origins of Object-Oriented Programming 7
Advantages of Object-Oriented Programming 7
C# and Object-Oriented Programming Concepts 8
Some C# code examples 8
Functional Programming vs Object-Oriented Programming 10
Code comparision 11
Simple example 11
Imperative OOP-style 11
Characteristics 11
Declarative style 12
More complex example: 13
Imperative OOP-style 13
Declarative style 14
Code comparison summary 15
OOP and FP Together 16
OOP + FP in Modern C# Design Patterns 16
How OOP + FP Shape Clean Architecture and DDD in Modern C# 17
Clean Architecture 17
OOP Contributions 17
FP Contributions 17
Domain Driven Design (DDD) 17
OOP Contributions 17
FP Contributions 18
Why This Hybrid Approach Works So Well 18

The Functional Programming Paradigm

Functional programming (FP) is a declarative programming paradigm that emphasizes describing what should be done, rather than how. Unlike the imperative paradigm, which includes OOP, FP focuses on the "what" rather than the "how." Combining paradigms can leverage their strengths.

The core characteristics of FP include:

- **Immutability**: Variables are assigned a value at declaration and do not change afterward, leading to more predictable and robust programs.
- **Higher-order functions**: Functions can be passed as parameters, returned, and stored as variables, enabling modular and reusable code.
- **Declarative programming**: Code consists of expressions that always evaluate to values, unlike statements that execute instructions.
- **Referential transparency** : Meaning that functions are pure, i.e. they have no side effects and always produce the same result for the same input values.
- **Recursion**: Used as an alternative to loops for iteration.
- **Pattern matching**: Efficient handling of different cases and types, which in C# has been improved with switch expressions.
- **Statelessness**: FP avoids shared state and mutable objects, facilitating parallelism and predictability.

Origins of Functional Programming

Functional programming (FP) has its origins in mathematical logic from the late 19th to mid-20th century, with significant contributions from Haskell Brooks Curry, Alonzo Church, Moses Schönfinkel, and Friedrich Frege.
The first programming languages to support FP were IPL (1956) and LISP (1958), though they were not entirely pure functional languages. Today, modern pure functional languages include Haskell, Elm, Elixir, and PureScript, while many languages like C#, JavaScript, Python, and Java are multiparadigm and incorporate functional concepts.

Advantages of Functional Programming

- **Concise and elegant code**: With less boilerplate, FP focuses on what should be done, making the code easier to understand and maintain.
- **Testability**: Pure functionality and referential transparency make the code easier to unit test and support test-driven development.
- **Robustness**: FP structures minimize errors, such as eliminating null references.
- **Predictability**: The code flow is linear and easy to follow, without complex exception handling that can cause unexpected behavior.

- **Better support for parallelism and concurrency**: Since FP is stateless, multiple instances can run simultaneously without shared state issues, which is beneficial in container and serverless environments.

Reduced code noise: A high signal-to-noise ratio in the code makes it easier to focus on business logic.

C# and Functional Programming Concepts

C# supports functional programming concepts within its multiparadigm design. It provides immutability through readonlyfields, record types, and the System.Collections.Immutable namespace.

Higher-order functions are enabled via delegates like Func and Action, allowing functions to be passed and returned as values.

With expression-based programming, features such as lambda expressions and LINQ let developers write concise, declarative code. Referential transparency is achieved by writing pure functions without side effects, while recursion can replace loops for iteration.

Modern C# also includes pattern matching with switch expressions and encourages stateless design, making it easier to build predictable, testable, and concurrent applications.

Some C# code examples

Immutability means that once a value or object is created, it cannot be changed, ensuring data stays consistent over time. Instead of modifying existing state, new values are produced, which makes programs more predictable and easier to reason about.

```
public record Person(string Name, int Age);

var person1 = new Person("Alice", 30);

// Records are immutable: you can't change Age directly.
// Instead, you create a new instance with 'with' expression:
var person2 = person1 with { Age = 31 };

Console.WriteLine(person1); // Person { Name = Alice, Age = 30 }
Console.WriteLine(person2); // Person { Name = Alice, Age = 31 }
```

Higher-Order Functions are functions that can take other functions as parameters or return them as results. In C#, this can be achieved using delegate types such as Func and Action. These delegates can be stored in variables and passed around the codebase, enabling modular and reusable code.

```
Func<int, int, int> add = (x, y) => x + y;

Func<Func<int, int, int>, int> applyTwice = f => f(2, 3) + f(4, 5);

int result = applyTwice(add);
Console.WriteLine(result); // (2+3) + (4+5) = 14
```

Expression-Based Programming means that code consists of expressions that always evaluate to values, unlike statements that execute instructions. In C#, this can be achieved using **lambda expressions** and **LINQ (Language Integrated Query)**. These tools allow concise and readable code that focuses on *what* should be done rather than *how*.

```
var numbers = new[] { 1, 2, 3, 4, 5 };

var squares = numbers.Select(n => n * n);

foreach (var s in squares)
    Console.WriteLine(s); // 1, 4, 9, 16, 25
```

Referential Transparency means that a function always produces the same result for the same input values and has no side effects. In C#, this can be achieved by writing **pure functions** that do not modify anything outside the function and always return the same result for the same inputs. This makes code more predictable and easier to test.

```
int Square(int x) => x * x;

Console.WriteLine(Square(5)); // Always 25
Console.WriteLine(Square(5)); // Always 25, no side effects
```

Recursion is a technique where a function calls itself to solve a problem. In C#, recursion can be used as an alternative to loops for iteration. Care must be taken, however, as recursion can lead to stack overflow if not handled properly.

```
int Factorial(int n)
{
    if (n == 0) return 1;
    return n * Factorial(n - 1);
}

Console.WriteLine(Factorial(5)); // 120
```

Pattern Matching in C# involves using **switch expressions** to handle different cases and types. With switch expressions, you can change the execution path based on the type of the object being examined and its properties. This makes code more concise and readable.

```
string DescribeShape(object shape) => shape switch
{
    Circle c => $"Circle with radius {c.Radius}",
    Rectangle r => $"Rectangle {r.Width}x{r.Height}",
    _ => "Unknown shape"
};

public record Circle(double Radius);
public record Rectangle(double Width, double Height);

Console.WriteLine(DescribeShape(new Circle(5)));
Console.WriteLine(DescribeShape(new Rectangle(3, 4)));
```

Statelessness means avoiding shared state and mutable objects. In C#, this can be achieved by using immutable data types and pure functions that do not modify anything outside the function. This facilitates parallelism and makes code more predictable.

```
// Pure function: no shared state, no side effects
int AddNumbers(int a, int b) => a + b;

var sum = AddNumbers(10, 20);
Console.WriteLine(sum); // 30
```

The Object-Oriented Programming Paradigm

Object-Oriented Programming (OOP) is an imperative programming paradigm that emphasizes describing *how* tasks should be performed through objects that encapsulate both data and behavior. Unlike the declarative paradigm of FP, OOP focuses on the *how* rather than the *what*. Combining paradigms can leverage their strengths. The core characteristics of OOP include:

- **Encapsulation**: Data and methods are bundled together inside objects, protecting internal state and exposing only necessary functionality.
- **Abstraction**: Complex systems are simplified by modeling classes and objects that represent real-world concepts.
- **Inheritance**: Classes can derive from other classes, reusing and extending functionality to promote code reuse and hierarchical design.
- **Polymorphism**: Objects can take on multiple forms, allowing methods to be overridden or interfaces to be implemented for flexible and extensible code.
- **Mutable State**: Objects often maintain internal state that can change over time, reflecting dynamic behavior.
- **Message Passing**: Objects interact by sending messages (method calls), enabling modular and distributed system design.
- **Iteration with Loops**: OOP typically uses loops and iterative constructs rather than recursion for repetition.

Origins of Object-Oriented Programming

OOP has its origins in the 1960s and 1970s, with early contributions from Ole-Johan Dahl and Kristen Nygaard, who developed **Simula** (1967), widely regarded as the first object-oriented language. Later, **Smalltalk** (1972) refined the paradigm, introducing pure object orientation. Today, mainstream languages such as **C#**, **Java**, **C++**, **Python**, are multiparadigm but strongly object-oriented, combining OOP with functional and procedural features.

Advantages of Object-Oriented Programming

- **Modularity**: Code is organized into classes and objects, making systems easier to structure and maintain.
- **Reusability**: Inheritance and polymorphism allow developers to reuse and extend existing code.
- **Maintainability**: Encapsulation ensures that changes to internal implementation do not affect external usage.
- **Scalability**: OOP's hierarchical design supports large-scale applications with complex relationships.
- **Intuitiveness**: OOP models real-world entities, making it easier to conceptualize and communicate system design.
- **Rich Ecosystem**: Most modern frameworks and libraries are designed with OOP principles, making integration straightforward.

C# and Object-Oriented Programming Concepts

C# is primarily an object-oriented language but also supports functional and procedural paradigms. Its OOP features include:

- **Classes and Objects**: The foundation of C#, encapsulating data and behavior.
- **Inheritance**: Classes can extend other classes, promoting reuse.
- **Interfaces and Abstract Classes**: Enable polymorphism and flexible design.
- **Properties and Fields**: Encapsulate state with controlled access.
- **Access Modifiers**: (public, private, protected) enforce encapsulation and control visibility.
- **Virtual and Override Methods**: Support polymorphism by allowing derived classes to redefine behavior.
- **Events and Delegates**: Facilitate message passing between objects.

Some C# code examples

Encapsulation with Classes

```
public class BankAccount
{
    private decimal balance;

    public void Deposit(decimal amount) => balance += amount;
    public void Withdraw(decimal amount) => balance -= amount;
    public decimal GetBalance() => balance;
}
```

Inheritance and Polymorphism

```
public class Animal
{
    public virtual void Speak() => Console.WriteLine("Some sound");
}

public class Dog : Animal
{
    public override void Speak() => Console.WriteLine("Woof!");
}

Animal pet = new Dog();
pet.Speak(); // Woof!
```

Abstraction with Interfaces

```
public interface IShape
{
    double Area();
}

public class Circle : IShape
{
    public double Radius { get; }
    public Circle(double radius) => Radius = radius;
    public double Area() => Math.PI * Radius * Radius;
}
```

Functional Programming vs Object-Oriented Programming

Aspect	Functional Programming	Object-Oriented Programming
Paradigm Type	Declarative – focuses on *what* should be done	Imperative – focuses on *how* tasks are performed
Core Unit	Functions	Objects (data + behavior)
State Management	Stateless, avoids shared mutable state	Stateful, objects maintain and update internal state
Immutability	Variables and data structures are immutable	Objects often rely on mutable fields and properties
Code Structure	Expressions that evaluate to values	Classes and methods that execute instructions
Key Features	Higher order functions, recursion, pattern matching, referential transparency	Encapsulation, inheritance, polymorphism, abstraction
Iteration	Recursion preferred over loops	Loops and iterative constructs are common
Origins	Rooted in mathematical logic	Originated in Simula
Languages	Pure FP: Haskell, Elm, Elixir, PureScript; Multiparadigm: C#, JavaScript, Python, Java	Strongly OOP: C#, Java, C++, Python;
Advantages	Concise, predictable, testable, robust, better for concurrency and parallelism	Modular, reusable, maintainable, scalable, intuitive for modeling real-world systems
C# Support	Records, LINQ, delegates (Func, Action), immutable collections, switch expressions	Classes, interfaces, inheritance, polymorphism, encapsulation, e vents, access modifiers

Code comparision

Let's compare **imperative OOP-style code** with a **functional declarative approach** in C#.

Simple example

In this example I will be filtering and transforming a list of numbers.

Imperative OOP-style

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6 };
        List<int> evenSquares = new List<int>();

        foreach (int n in numbers)
        {
            if (n % 2 == 0) // imperative condition
            {
                int square = n * n;
                evenSquares.Add(square);
            }
        }

        foreach (int result in evenSquares)
        {
            Console.WriteLine(result);
        }
    }
}
```

Characteristics

- Explicit loops (foreach).
- Manual state management (evenSquares list).
- Step-by-step instructions on *how* to do the task.

Declarative style

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        var numbers = new[] { 1, 2, 3, 4, 5, 6 };

        var evenSquares = numbers
            .Where(n => n % 2 == 0)   // declarative filtering
            .Select(n => n * n);      // declarative transformation

        foreach (var result in evenSquares)
        {
            Console.WriteLine(result);
        }
    }
}
```

Characteristics

- Uses **higher-order functions** (Where, Select).
- No explicit mutable state.
- Focuses on *what* should be done (filter evens, square them), not *how*.

More complex example:

Let's go deeper with a **more complex example** that compares **imperative OOP-style code** (using for/foreach) with a **functional declarative approach** (using LINQ's .Aggregate).

We'll solve a realistic problem: **calculate the total price of items in a shopping cart, applying discounts and taxes**.

Imperative OOP-style

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        var cart = new List<Item>
        {
            new Item("Book", 12.99m, 0.10m),    // 10% discount
            new Item("Pen", 1.50m, 0.0m),       // no discount
            new Item("Notebook", 5.00m, 0.05m) // 5% discount
        };

        decimal taxRate = 0.25m; // 25% VAT
        decimal total = 0;

        foreach (var item in cart)
        {
            decimal discountedPrice = item.Price * (1 - item.Discount);
            decimal taxedPrice = discountedPrice * (1 + taxRate);
            total += taxedPrice;
        }

        Console.WriteLine($"Total: {total:F2}");
    }
}

public class Item
{
    public string Name { get; }
    public decimal Price { get; }
    public decimal Discount { get; }

    public Item(string name, decimal price, decimal discount)
    {
        Name = name;
        Price = price;
        Discount = discount;
    }
}
```

Characteristics

- Explicit loop (foreach).
- Manual accumulation (total += taxedPrice).
- Step-by-step instructions on *how* to compute the result.

Declarative style

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        var cart = new List<Item>
        {
            new Item("Book", 12.99m, 0.10m),
            new Item("Pen", 1.50m, 0.0m),
            new Item("Notebook", 5.00m, 0.05m)
        };

        decimal taxRate = 0.25m;

        decimal total = cart.Aggregate(0m, (acc, item) =>
        {
            var discounted = item.Price * (1 - item.Discount);
            var taxed = discounted * (1 + taxRate);
            return acc + taxed;
        });

        Console.WriteLine($"Total: {total:F2}");
    }
}

public record Item(string Name, decimal Price, decimal Discount);
```

Characteristics

- Uses .Aggregate to fold the list into a single value.
- No explicit mutable state (total is computed in one expression).
- Declarative: describes *what* transformation to apply, not *how* to iterate.

Code comparison summary

In the first example, the imperative version used explicit loops and mutable state, while the functional version used LINQ's Where and Select to express the logic as a transformation pipeline without managing iteration manually.

In the second example demonstrated a more complex scenario: calculating the total cost of items in a shopping cart with discounts and taxes. The imperative version relied on a foreach loop and step-by-step accumulation, whereas the functional version used LINQ's Aggregate to fold the collection into a single value. This highlighted how functional code can express complex business logic more concisely and with fewer mutable variables.

OOP and FP Together

Modern C# design patterns often work so well precisely because the language blends **object-oriented** and **functional** paradigms into a Hybrid Approach. Each paradigm contributes strengths that make today's architectural patterns more expressive, safer, and easier to maintain.

OOP + FP in Modern C# Design Patterns

Builder Pattern (Modern Fluent Builders)

FP influence: fluent, expression-based APIs; immutable "with" semantics **OOP influence:** encapsulation, object construction patterns
Modern builders often return new immutable objects rather than mutating internal state, blending OOP structure with FP safety.

Inversion of Control (IoC) / Dependency Injection

FP influence: stateless services, pure functions, declarative registration **OOP influence:** interfaces, polymorphism, dependency inversion
IoC containers work because OOP provides abstraction boundaries, while FP encourages stateless, predictable service lifetimes.

Entity Framework Core Query Pipeline

FP influence: expression trees, immutable query objects, declarative transformations **OOP influence:** entity modeling, navigation properties, inheritance
EF Core's LINQ provider is essentially a functional query engine wrapped in an OOP domain model.

Pipeline Delegate Pattern (Middleware / Delegating Handlers)

FP influence: function composition, higher-order functions, immutability **OOP influence:** encapsulated components, interfaces, chain-of-responsibility structure
The Pipeline Delegate Pattern—used heavily in **ASP.NET Core middleware**, **HttpClient handlers**, and **custom processing pipelines**—is one of the clearest examples of OOP and FP working together:
From **FP**, it borrows the idea of composing functions into a pipeline, where each step receives input and returns output without mutating shared state.
From **OOP**, it uses classes, interfaces, and encapsulated handlers to structure each pipeline stage.
Each middleware component is effectively a **higher-order function** that wraps the next delegate, creating a clean, composable chain of behaviors. This makes pipelines easy to extend, test, and reason about.

Modern C# patterns succeed because they combine **OOP's structural modeling** (classes, interfaces, encapsulation) with **FP's declarative, immutable, and composable style**. This synergy enables safer APIs, clearer domain models, more predictable behavior, and powerful abstractions like LINQ, DI containers, and EF Core's query pipeline.

How OOP + FP Shape Clean Architecture and DDD in Modern C#

Modern architectural styles like **Clean Architecture** and **Domain-Driven Design (DDD)** rely heavily on the strengths of both **object-oriented** and **functional** paradigms. C#'s ability to blend these paradigms is a major reason why it excels in large, maintainable enterprise systems.

Clean Architecture

Clean Architecture emphasizes **separation of concerns**, **pure business logic**, and **independence from frameworks**.

OOP Contributions

- **Entities and Use Cases** modeled as classes with clear boundaries
- **Interfaces** to decouple layers (e.g., repositories, services, gateways)
- **Dependency Inversion** to keep business logic independent of infrastructure

FP Contributions

- **Pure functions** inside use cases reduce side effects and make business rules predictable
- **Immutability** ensures stable data flow between layers
- **LINQ pipelines** allow declarative transformations inside application services
- **Option/Result types** (or equivalents) reduce nulls and exceptions, improving reliability

Together, OOP provides the *structure*, while FP provides the *behavioral safety* that makes Clean Architecture robust and testable.

Domain Driven Design (DDD)

DDD focuses on modeling complex domains using rich domain models and ubiquitous language.

OOP Contributions

- **Aggregates, Entities, Value Objects** map naturally to classes and interfaces
- **Encapsulation** protects invariants inside aggregates
- **Polymorphism** supports domain variation and extensibility

FP Contributions

- **Value Objects** become safer when immutable
- **Domain rules** expressed as pure functions are easier to test and reason about
- **Pattern matching** simplifies domain logic branching
- **Declarative transformations** (LINQ, expression trees) make domain queries expressive and intention-revealing

In DDD, OOP models the domain structure, while FP ensures domain logic is **pure, predictable, and side-effect free**.

Why This Hybrid Approach Works So Well

C#'s combination of paradigms enables:

- **Rich domain models (OOP)**
- **Safe, predictable business logic (FP)**
- **Declarative data processing (LINQ)**
- **Framework independence (Clean Architecture)**
- **Explicit domain behavior (DDD)**
- **Testability and maintainability across layers**

This synergy is why modern C# systems often feel both **structured** and **expressive**, combining the best of both worlds.